

bdd

(<https://github.com/e-t-u/bdd>)

Bit Stream Manipulation Command & Native
Library

For Unix/Linux/POSIX Command Line

Re-engineered in Modern Rust (v0.3.0)

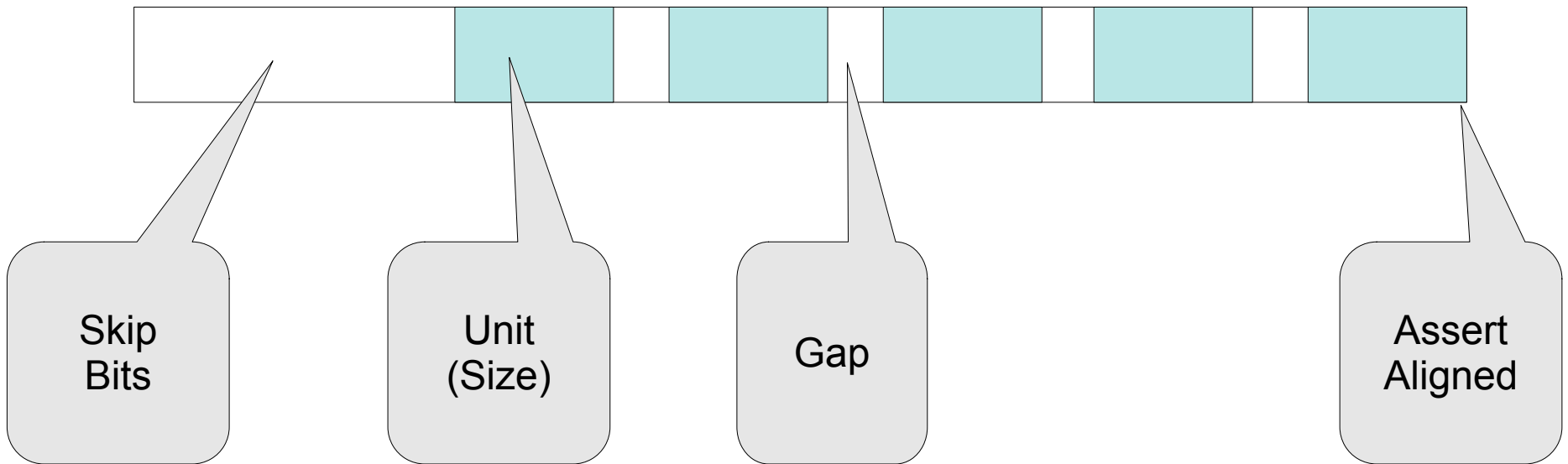
Multi-Gigabit Throughput with 64-Bit Sub-Byte
Precision

Open Source (GPL-3.0)

By

Esa Turtiainen

A Bit Stream

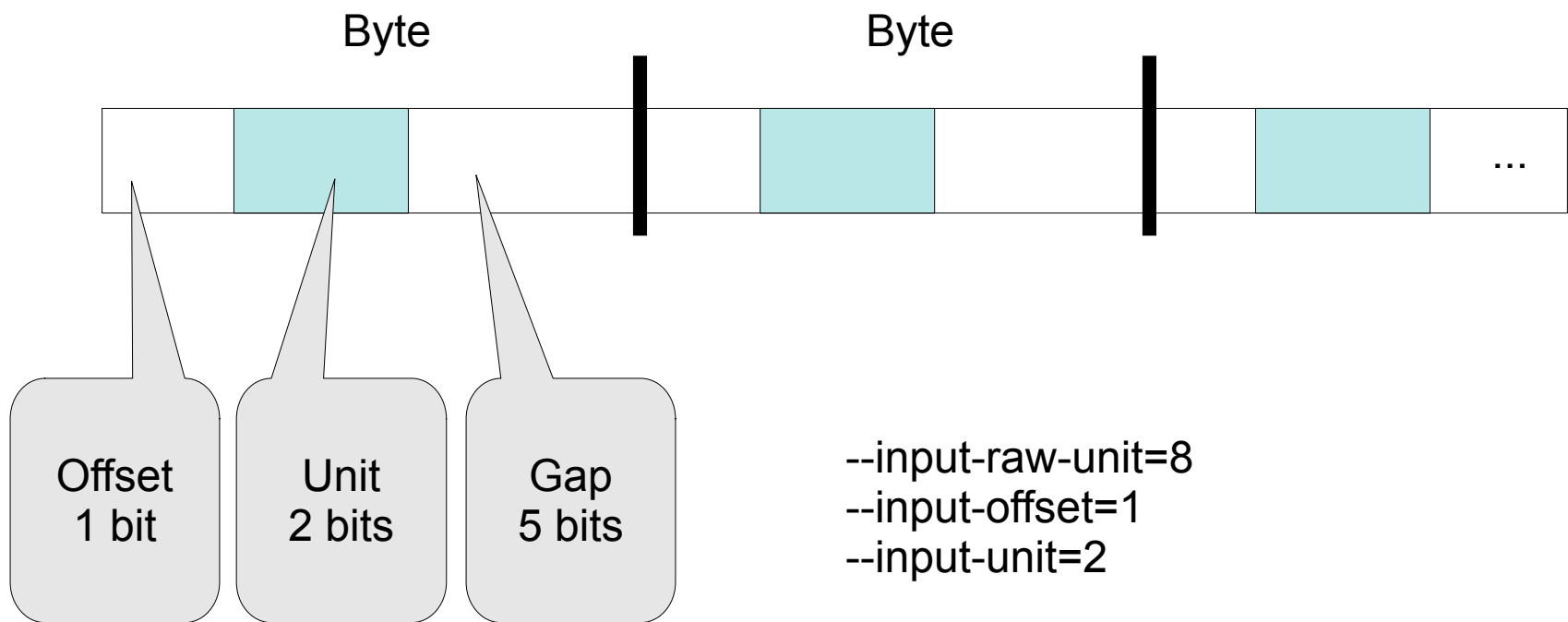


All measures in bits, not bytes (Full 64-bit integer range) Supports scale suffixes: K, M, G, T, KiB, GiB, B
Supports size math: $1920 \times 1080 \times 24$, 1000000×24

e.g.
`--input-skip-bits=128`
`--input-unit=7`
`--input-gap=3`
`--input-assert-aligned`

Gives an error message if not aligned to a byte

Raw Units & Containers



Raw Unit is the repeating outer container, frame, or stride (e.g. 8, 16, 32 bits)

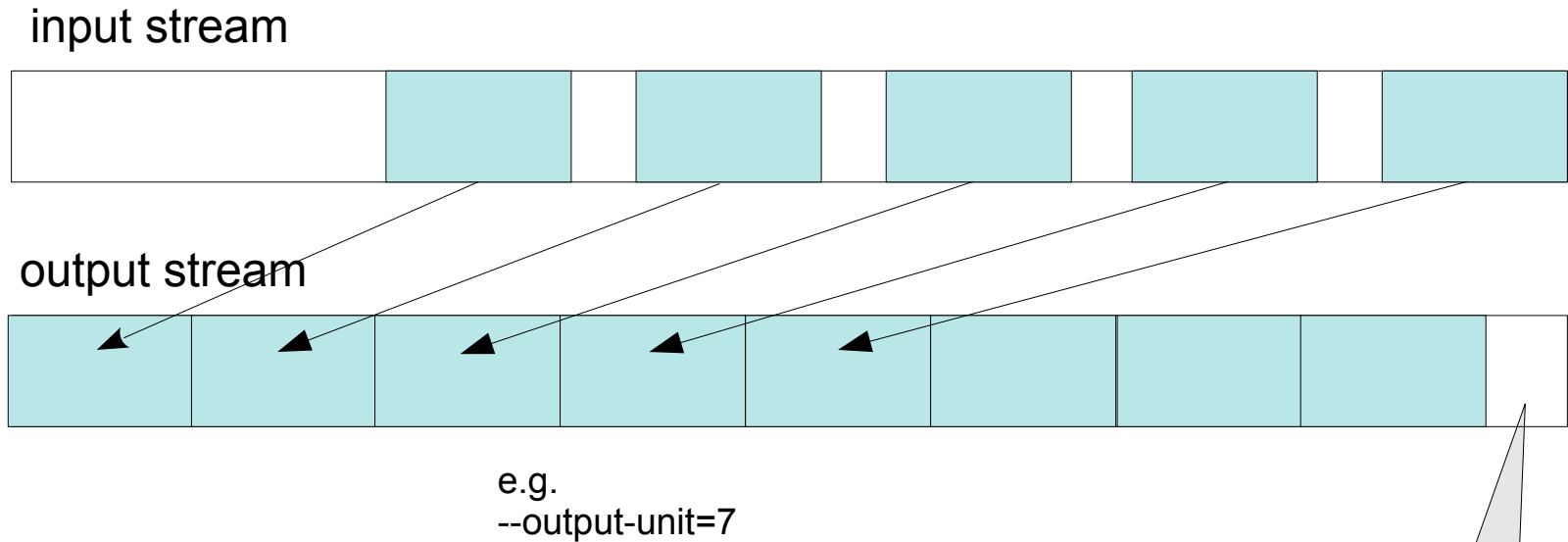
--input-offset sets active unit offset inside container

bdd derives skip & trailing gap ($R - (O + U)$)

automatically

Eliminates tedious manual arithmetic on structured bitstreams!

Output Bit Stream



If the size of the output unit is bigger
than the input unit:

Left bits are truncated

If the size of the output unit is smaller
than the input unit:

Left bits are filled with zeros

Unit is handled as if
it was an integer

Last Byte
Filled with
Zeros

Example, Rearrange Bit Streams

- Take most significant bit of a byte stream
 - `bdd --input-unit=1 --input-gap=7 \`
`--output-unit=8`
 - examples assume that the input is piped to the bdd like “cat file | bdd”, you can also use `--input-file`
 - 1 bit will be filled to 8 bits: most significant bits = 0
 - Output byte stream of values 0x00 and 0x01
- Pack least significant bit of bytes to a bit stream
 - `bdd --input-skip-bits=7 --input-unit=1 --`
`input-gap=7 --output-unit=1`
(or with raw units: `bdd --input-raw-unit=8 --input-offset=7 --input-unit=1 --output-unit=1`)
 - If input is 9 bytes all bits ones, the output will be two bytes 0xFF and 0x80

“Fake” Bit Streams

- Input:
 - Zeros
 - Ones
 - Random
 - /dev/urandom
 - Counter
 - Starts from zero
 - Every unit previous plus one
- Output:
 - Print unit as integer
 - one integer at line
 - meant for programs
 - Print unit as hex
 - human-readable lines
 - Print unit as bits
 - human-readable lines

Print unit as JSON (--
output-json)

NDJSON record per
tuple (for i.e. Python

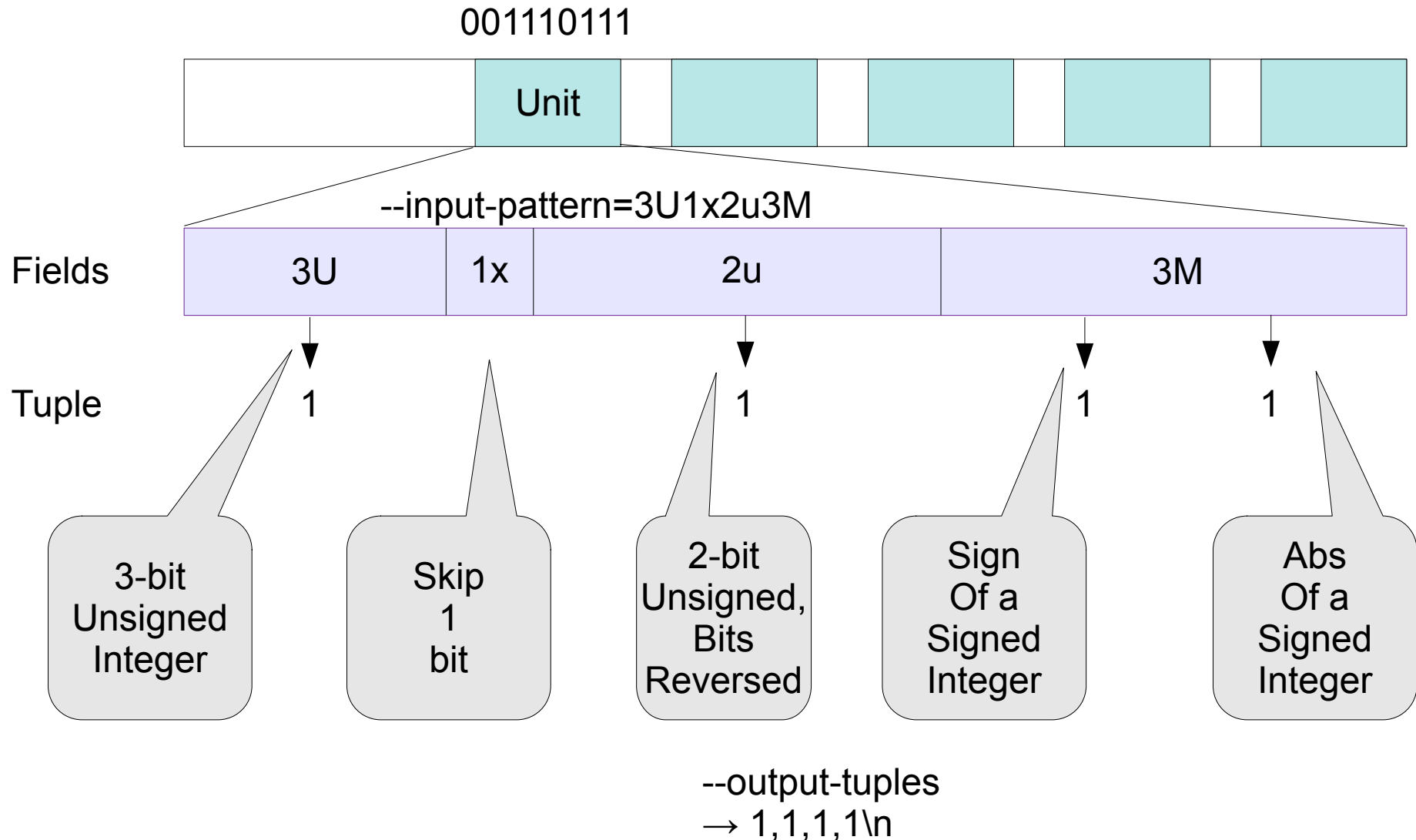
Default Unit Size & 64-Bit Seeking

- Rule to remember: `bdd < a > b` copies 8-bit bytes unchanged (byte-exact cat / dd)
- Default input unit: 8 bits | Default output unit: 8 bits
- Full 64-bit architecture: offsets & units up to 2.3 Exabytes ($2^{64}-1$ bits)
- Fast $O(1)$ filesystem seek: jumps gigabytes in microseconds on disk files
- Automatic fallback to sequential streaming on pipes, FIFOs, and sockets (`--no-seek`)

Tuple

- Tuple is a unit that is split into fields
- It is possible to give fields types so that e.g. some bits are interpreted as signed integers or floating point numbers

Tuple Unpack



Pattern Characters (Input)

- nU / nB — Unsigned integer / Byte (standard big-endian)
- nu / nb — Unsigned integer (reversed bit order)
- nS / ns — Two's complement signed integer (returns negative values)
- nM / nm — Sign-Split integer returning two fields: [sign, abs(val)]
- nx — Skip / discard n bits from input without placing in tuple
- Pattern Repetition Multipliers: 4*8B, 10*16H, 2*(4U4u)

“Fake” Tuple Streams

- `--input-tuples` reads human-readable tuples directly from the input stream
 - `--output-tuples` writes human-readable tuples directly to the output stream
 - Comma separated fields, one line per tuple:
 - `1,1.2,"abba"`
- `--output-json` writes newline-delimited JSON (NDJSON)
- `--output-csv` writes CSV (with optional `--csv-header`)
- `--output-visual` renders colorized ANSI field slices

Example, Look inside a Bit Stream

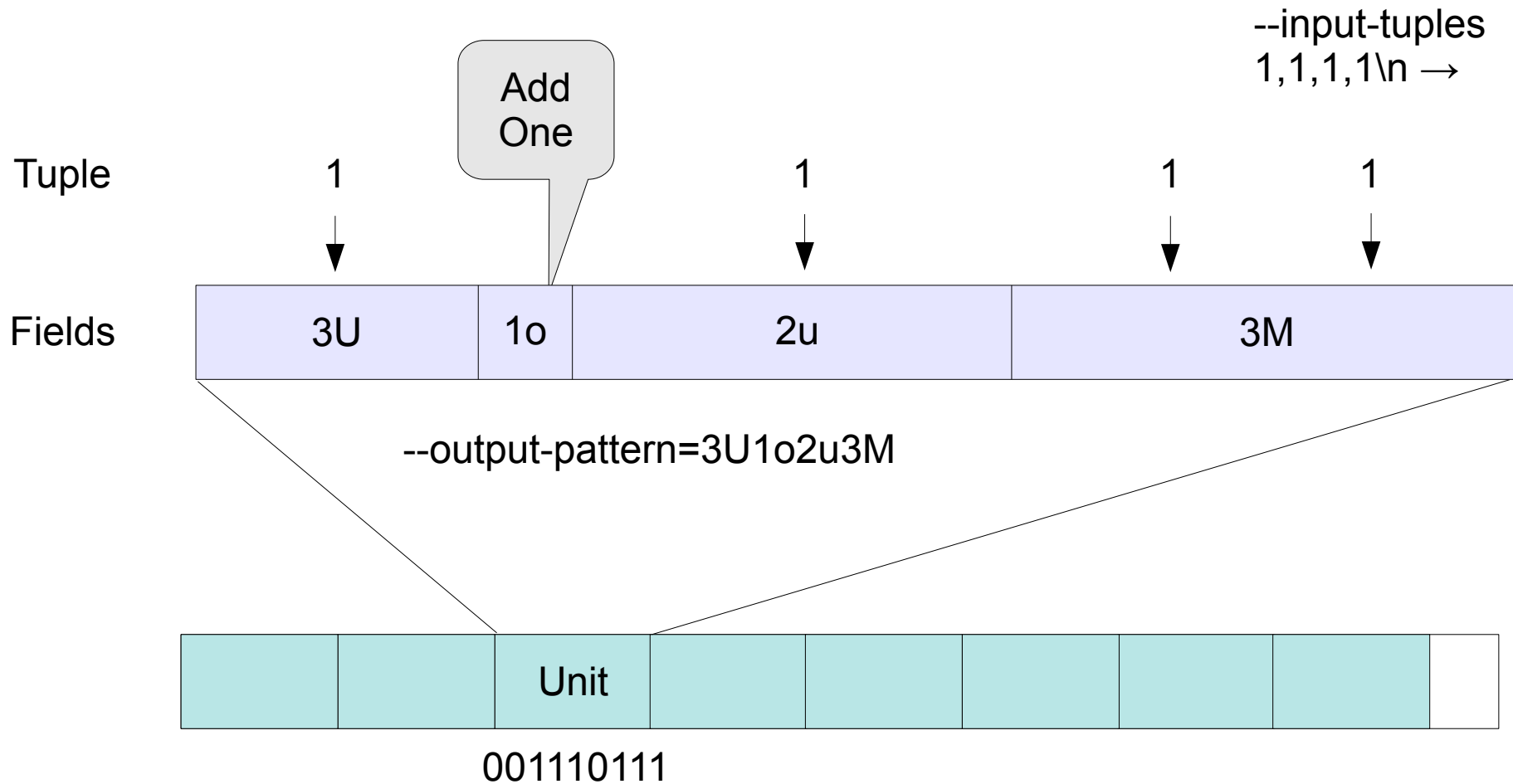
- Interpret units to human-readable lines:
 - `bdd --input-pattern=3U1x2u3M --output-tuples`
 - Prints result as comma separated list, one unit at a line, e.g.

```
1,1,1,1  
3,2,0,2  
...
```

Floating Point & AI Tensor Formats

- IEEE 754 Floats: 32F/32f (single), 64D/64d (double), 16H/16h (half FP16)
- Google Brain Bfloat16: 16Y / 16y (16 bits)
- OCP FP8 (E4M3FN): 8E / 8e (NVIDIA Hopper H100, 8 bits)
- OCP FP8 (E5M2): 8Q / 8q (NVIDIA Ada Lovelace, 8 bits)
- OCP Microscaling FP6: 6E / 6e (E3M2, 6 bits)
- NVIDIA Blackwell NVFP4: 4E / 4e (E2M1, 4 bits)

Tuple Pack



Pattern Characters (Output)

- Output patterns have some special characters:
- z – Fill bits with zeros
- o – Fill bits with ones
- r – Fill bits with random bits (/dev/urandom)

Examples, Pack from Text

- Build a hex number from integers:

- `echo -en "1,2\n3,4" | bdd --input-tuples \`
`--output-pattern=4U4U | od -t x1`

- Prints (hex) 12 34

- Build a stream of floating point numbers:

- `echo -en "1.0\n2.0\n" | \`
`bdd --input-tuples --output-pattern=64D`

Example, Combined

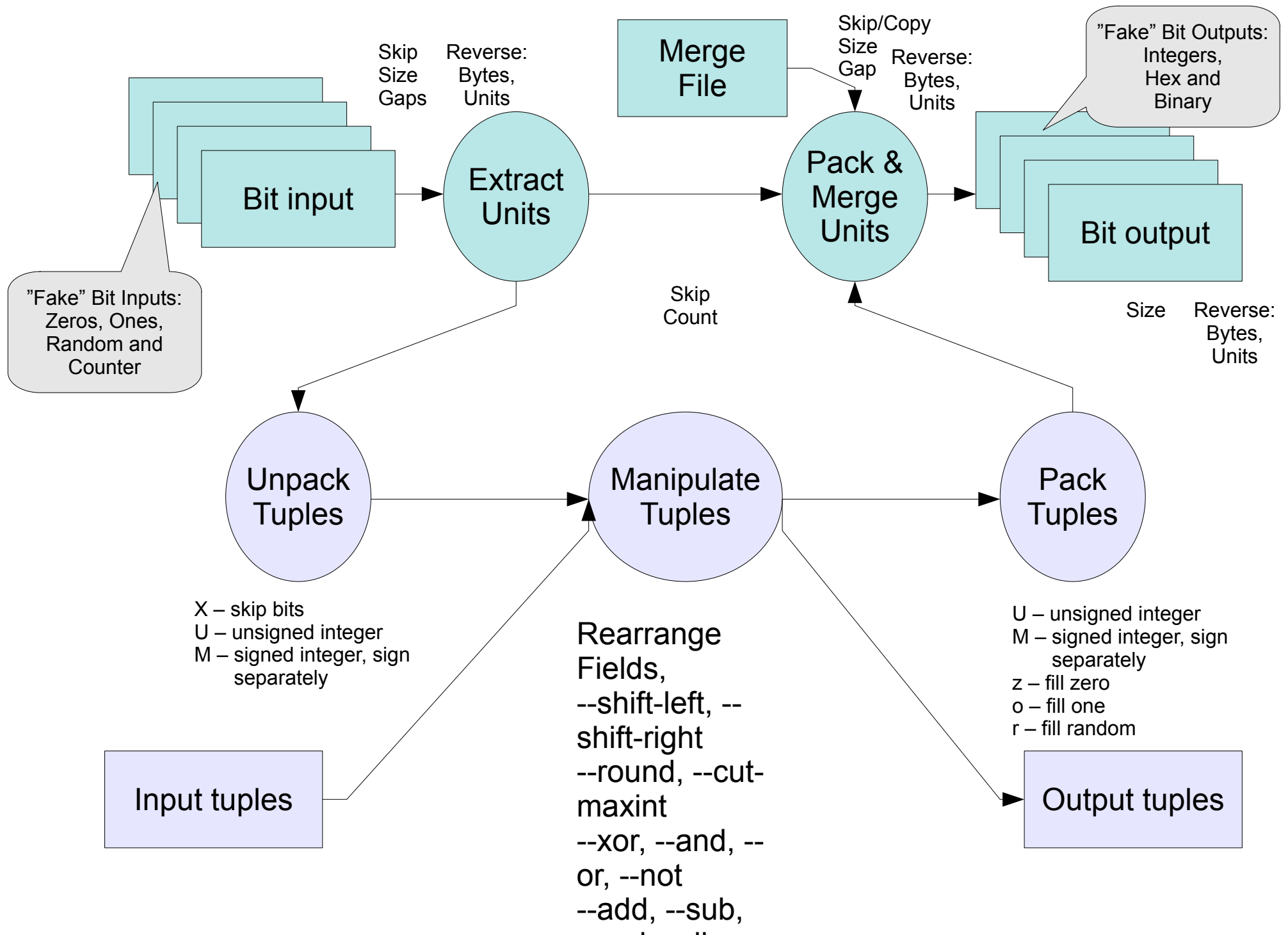
```
→ echo 1,1,1,1 | \  
  bdd --input-tuples --output-pattern=3U1o2u3M | \  
  bdd --input-pattern=3U1x2u3M --output-tuples
```

- Prints:

- 1,1,1,1

- 0,0,0,0

- 9-bit output pattern outputs two bytes and when read, the second is read as all-zeros



Ordered Pipeline Transformations

- Manipulators execute in exact command-line order specified:
- Arithmetic: --add, --sub, --mul, --div, --mod, --abs, --sign
- Bitwise: --shift-left, --shift-right, --xor, --and, --or, --not
- Bounds & Rounding: --round, --cut-maxint=F,MAX,MODE
(modes: saturate, wrap, drop, trunc, floor, ceil, round, round_ties_even)
- Predicate Filtering: --filter="FIELD,OP,VALUE"

Reversing Bits

- Reversing bits can be quite a mess...
- bdd implements
 - Reversing bits of every byte read
 - Reversing bits of every unit read
 - Matching patterns in reverse bit order
 - Reversing bits of every unit written
 - Reversing bits of every byte written
- --in/output-little-endian is a synonym for combined reversing of both bytes and unit

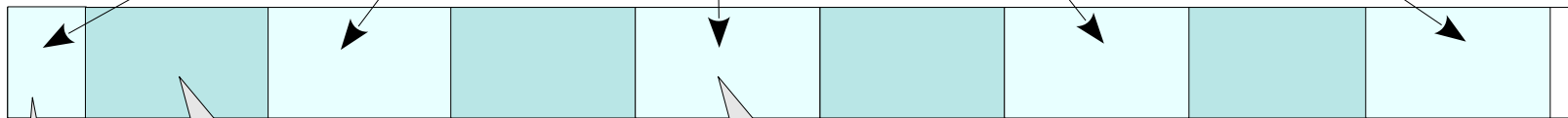
Merge Bit Stream

Merge file options
are mostly identical
to input file options:
skip, gap, reverse

Merge stream



Output stream



Normal
Unit

Merge
Unit

“Copy
First”

A merge unit is written after each normal unit
It is an error if merge file ends too early
Output will have exactly as many normal and merge units

Example, Merge

- Print first 16 bytes of /etc/passwd file as hex preceded by a three-digit hex address of the byte:

```
→ bdd --input-counter --count=16 \  
    --input-unit=12 --output-unit=12 \  
    --merge-file=/etc/passwd \  
    --output-hex
```

```
000 72 001 6f 002 6f 003 74 004 3a 005 78 006 3a 007 30  
008 3a 009 30 00a 3a 00b 72 00c 6f 00d 6f 00e 74 00f 3a
```

Modern AI Quantized Weight Slicing

- Native Sub-Byte & Microscaling AI Formats:
NVIDIA Blackwell 4-bit NVFP4 (4E4E), OCP 6-bit FP6 (4*6E), FP8 (8E/8Q), BF16 (16Y)
- Instant $O(1)$ Seeking into Large Weight Files:
Directly seek into HuggingFace Safetensors and GGUF files in microseconds without PyTorch/CUDA
- Weight Inspection & Decoding:
`bdd --input-file=model.safetensors --input-skip-bits=${OFFSET}B --input-pattern=8E --output-json`
- Sparsity & Outlier Detection:

Real-World Multimedia Slicing Recipes

- MPEG Audio (MP3) Frame Headers:
Unaligned 32-bit header unpacked with single command:
`11U2U2U1U4U2U1U1U2U2U1U1U2U`
- MPEG-TS 188-byte Packet Containers:
Slices 13-bit PID: `bdd --input-raw-unit=1504 --input-offset=11 --input-unit=13 --output-unit=13`
- RIFF WAV Stereo Audio & Demuxing:
Splits interleaved 16-bit stereo PCM into left & right mono channels: `--demux-files=l.raw,r.raw`
- Studio 24-bit PCM Audio:
Reads 24S signed audio and downsamples to

Performance & Programmatic Interfaces

- Multi-Gigabit Modern Rust Engine:
 - Hardware 64-bit bit reversal: 16 Gbps (direct CPU register acceleration)
 - Bignum 1024-bit packing/unpacking: 5.27 Gbps
 - Synthetic linear bit streaming: 200+ Mbps
- Native C-ABI Shared Library (libbdd.so & include/bdd.h):

```
bdd_unpack_u64("4U4U", unit, fields, 2);  
bdd_decode_fp8_e4m3(bits);  
bdd_decode_fp4_e2m1(bits);
```
- Zero-Dependency Python Wrapper